

Lab 8: RDP Parser for C Grammar

Date: 27/03/2026

6CSEC2@debian:~/Documents/230905252/Lab8\$ cat parser.c

```
#include "lexer.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    FILE *fp;
    Token lookahead;
    int row;
    int col;
    char dbuf[16];
    int errorFound;
} Parser;

const char *FOLLOW_program[] = { "EOF", NULL };
const char *FOLLOW_declarations[] = { "ID", "if", "}", NULL };
const char *FOLLOW_datatype[] = { "ID", NULL };
const char *FOLLOW_identifier_list[] = { ";", NULL };
const char *FOLLOW_id_list_prime[] = { ";", NULL };
const char *FOLLOW_id_list_double_prime[] = { ";", NULL };
const char *FOLLOW_statement_list[] = { "}", NULL };
const char *FOLLOW_statement[] = { "ID", "if", "}", NULL };
const char *FOLLOW_assign_stat[] = { ";", NULL };
const char *FOLLOW_expn[] = { ")", ";", NULL };
const char *FOLLOW_eprime[] = { ")", ";", NULL };
const char *FOLLOW_simple_expn[] = { ")", ";", "==", "!=", "<=", ">=", "<", ">", NULL };
const char *FOLLOW_seprime[] = { ")", ";", "==", "!=", "<=", ">=", "<", ">", NULL };
const char *FOLLOW_term[] = { "<=", ">=", "<", ">", NULL };
const char *FOLLOW_tprime[] = { "<=", ">=", "<", ">", NULL };
const char *FOLLOW_factor[] = { ";", "==", "!=", "<=", ">=", "<", ">", NULL };
const char *FOLLOW_decision_stat[] = { ";", NULL };
const char *FOLLOW_dprime[] = { ";", NULL };
const char *FOLLOW_relop[] = { "ID", "NUM", NULL };
const char *FOLLOW_addop[] = { "ID", "NUM", NULL };
const char *FOLLOW_mulop[] = { "ID", "NUM", NULL };

typedef struct {
    const char *nonterminal;
    const char **followSet;
} FollowEntry;

FollowEntry followTable[] = {
    { "program", FOLLOW_program },
    { "declarations", FOLLOW_declarations },
    { "datatype", FOLLOW_datatype },
```

```

    { "identifier_list", FOLLOW_identifier_list },
    { "id_list_prime", FOLLOW_id_list_prime },
    { "id_list_double_prime", FOLLOW_id_list_double_prime },
    { "statement_list", FOLLOW_statement_list },
    { "statement", FOLLOW_statement },
    { "assign_stat", FOLLOW_assign_stat },
    { "expn", FOLLOW_expn },
    { "eprime", FOLLOW_eprime },
    { "simple_expn", FOLLOW_simple_expn },
    { "seprime", FOLLOW_seprime },
    { "term", FOLLOW_term },
    { "tprime", FOLLOW_tprime },
    { "factor", FOLLOW_factor },
    { "decision_stat", FOLLOW_decision_stat },
    { "dprime", FOLLOW_dprime },
    { "relop", FOLLOW_relop },
    { "addop", FOLLOW_addop },
    { "mulop", FOLLOW_mulop },
    { NULL, NULL }
};

const char **getFollowSet(const char *nonterminal) {
    if (!nonterminal) return NULL;
    for (int i = 0; followTable[i].nonterminal != NULL; i++) {
        if (strcmp(followTable[i].nonterminal, nonterminal) == 0)
            return followTable[i].followSet;
    }
    return NULL;
}

int inFollowSet(const char *token, const char **followSet) {
    for (int i = 0; followSet[i] != NULL; i++) {
        if (strcmp(token, followSet[i]) == 0) return 1;
    }
    return 0;
}

void recover(Parser *p, const char **followSet) {
    while (strcmp(p->lookahead.token_name, "EOF") != 0 &&
        !inFollowSet(p->lookahead.token_name, followSet)) {
        p->lookahead = getNextToken(p->fp, &p->row, &p->col, p->dbuf);
    }
}

void syntaxError(Parser *p, const char *expected, const char *nonterminal) {
    p->errorFound = 1;
    printf("\nSyntax Error at row %d, col %d\n", p->lookahead.row, p->lookahead.col);
    printf("Expected: %s | Found: %s\n", expected, p->lookahead.token_name);

    if (nonterminal) {
        const char **fs = getFollowSet(nonterminal);
        if (fs) recover(p, fs);
    } else {
        p->lookahead = getNextToken(p->fp, &p->row, &p->col, p->dbuf);
    }
}

void match(Parser *p, const char *expected) {

```

```

    if (strcmp(p->lookahead.token_name, expected) == 0) {
        if (!p->errorFound)
            printf("<%s,%d,%d>", p->lookahead.token_name, p->lookahead.row, p->lookahead.col);
        p->lookahead = getNextToken(p->fp, &p->row, &p->col, p->dbuf);
    } else {
        syntaxError(p, expected, NULL);
    }
}

void program(Parser *p);
void declarations(Parser *p);
void datatype(Parser *p);
void identifier_list(Parser *p);
void id_list_prime(Parser *p);
void id_list_double_prime(Parser *p);
void statement_list(Parser *p);
void statement(Parser *p);
void assign_stat(Parser *p);
void decision_stat(Parser *p);
void dprime(Parser *p);
void looping_stat(Parser *p);
void expn(Parser *p);
void eprime(Parser *p);
void simple_expn(Parser *p);
void seprime(Parser *p);
void term(Parser *p);
void tprime(Parser *p);
void factor(Parser *p);
void relop(Parser *p);
void addop(Parser *p);
void mulop(Parser *p);

void program(Parser *p) {
    match(p, "main");
    match(p, "(");
    match(p, ")");
    match(p, "{");
    declarations(p);
    statement_list(p);
    match(p, "}");
}

void declarations(Parser *p) {
    if (strcmp(p->lookahead.token_name, "int") == 0 || strcmp(p->lookahead.token_name, "char") == 0) {
        datatype(p);
        identifier_list(p);
        match(p, ";");
        declarations(p);
    }
}

void datatype(Parser *p) {
    if (strcmp(p->lookahead.token_name, "int") == 0) match(p, "int");
    else if (strcmp(p->lookahead.token_name, "char") == 0) match(p, "char");
    else syntaxError(p, "int or char", "datatype");
}

```

```
void identifier_list(Parser *p) {
    match(p, "ID");
    id_list_prime(p);
}

void id_list_prime(Parser *p) {
    if (strcmp(p->lookahead.token_name, "[") == 0) {
        match(p, "[");
        match(p, "NUM");
        match(p, "]");
        id_list_double_prime(p);
    } else if (strcmp(p->lookahead.token_name, ",") == 0) {
        match(p, ",");
        identifier_list(p);
    }
}

void id_list_double_prime(Parser *p) {
    if (strcmp(p->lookahead.token_name, ",") == 0) {
        match(p, ",");
        identifier_list(p);
    }
}

void statement_list(Parser *p) {
    if (strcmp(p->lookahead.token_name, "ID") == 0 ||
        strcmp(p->lookahead.token_name, "if") == 0) {
        statement(p);
        statement_list(p);
    }
}

void statement(Parser *p) {
    if (strcmp(p->lookahead.token_name, "ID") == 0) {
        assign_stat(p);
        match(p, ";");
    } else if (strcmp(p->lookahead.token_name, "if") == 0) {
        decision_stat(p);
    } else {
        syntaxError(p, "statement", "statement");
    }
}

void assign_stat(Parser *p) {
    match(p, "ID");
    if (strcmp(p->lookahead.token_name, "[") == 0) {
        match(p, "[");
        match(p, "NUM");
        match(p, "]");
    }
    match(p, "=");
    expn(p);
}

void expn(Parser *p) {
    simple_expn(p);
    eprime(p);
}
```

```
void eprime(Parser *p) {
    char *tn = p->lookahead.token_name;
    if (strcmp(tn, "==") == 0 || strcmp(tn, "!=") == 0 || strcmp(tn, "<=") == 0
    ||
        strcmp(tn, ">=") == 0 || strcmp(tn, ">") == 0 || strcmp(tn, "<") == 0) {
        relop(p);
        simple_expn(p);
    }
}

void simple_expn(Parser *p) {
    term(p);
    seprime(p);
}

void seprime(Parser *p) {
    if (strcmp(p->lookahead.token_name, "+") == 0 || strcmp(p-
>lookahead.token_name, "-") == 0) {
        addop(p);
        term(p);
        seprime(p);
    }
}

void term(Parser *p) {
    factor(p);
    tprime(p);
}

void tprime(Parser *p) {
    if (strcmp(p->lookahead.token_name, "*") == 0 ||
        strcmp(p->lookahead.token_name, "/") == 0 ||
        strcmp(p->lookahead.token_name, "%") == 0) {
        mulop(p);
        factor(p);
        tprime(p);
    }
}

void factor(Parser *p) {
    if (strcmp(p->lookahead.token_name, "ID") == 0) {
        match(p, "ID");
        if (strcmp(p->lookahead.token_name, "[") == 0) {
            match(p, "[");
            match(p, "NUM");
            match(p, "]");
        }
    }
    else if (strcmp(p->lookahead.token_name, "NUM") == 0) match(p, "NUM");
    else syntaxError(p, "ID or NUM", "factor");
}

void decision_stat(Parser *p) {
    match(p, "if");
    match(p, "(");
    expn(p);
    match(p, ")");
    match(p, "{");
    statement_list(p);
}
```

```

    match(p, "}");
    dprime(p);
}

void dprime(Parser *p) {
    if (strcmp(p->lookahead.token_name, "else") == 0) {
        match(p, "else");
        match(p, "{");
        statement_list(p);
        match(p, "}");
    }
}

void relop(Parser *p) {
    match(p, p->lookahead.token_name);
}

void addop(Parser *p) {
    match(p, p->lookahead.token_name);
}

void mulop(Parser *p) {
    match(p, p->lookahead.token_name);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 0;
    }
    FILE *fp = fopen(argv[1], "r");
    if (!fp) {
        printf("Error: Cannot open %s\n", argv[1]);
        return 0;
    }
    Parser parser;
    parser.fp = fp;
    parser.row = 1;
    parser.col = 0;
    parser.errorFound = 0;
    parser.lookahead = getNextToken(fp, &parser.row, &parser.col, parser.dbuf);

    program(&parser);

    if (!parser.errorFound && strcmp(parser.lookahead.token_name, "EOF") == 0)
        printf("\nParsing Successful\n");

    fclose(fp);
    return 0;
}

```

Input 1: (with error)

6CSEC2@debian:~/Documents/230905252/Lab8\$ cat example.c

```

#include <stdio.h>
// Example C code to test the lexer and parser
main() {
    int a, b, arr[50], result;
    char flag, symbol;

```

```

a 10;
b = 20 + 5 * ;

if (a <= b) {
    result = a * b / 2;
    arr[1] = result % 3;
} else {
    result = 0;
}
6CSEC2@debian:~/Documents/230905252/Lab8$ ./exe/parser example.c
<main,3,1><(,3,5><),3,6><{,3,8><int,4,5><ID,4,9><,,4,10><ID,4,12><,,4,13><ID,4,15><[,4,18><NUM,4,19><],4,21><,,4,22><ID,4,24><;,4,30><char,5,5><ID,5,10><,,5,14><ID,5,16><;,5,22><ID,7,5>
Syntax Error at row 7, col 7
Expected: = | Found: NUM

Syntax Error at row 7, col 9
Expected: ID or NUM | Found: ;

Syntax Error at row 8, col 18
Expected: ID or NUM | Found: ;

Syntax Error at row 15, col 1
Expected: } | Found: EOF

```

Input 2: (without error)

```

6CSEC2@debian:~/Documents/230905252/Lab8$ cat example.c
#include <stdio.h>
// Example C code to test the lexer and parser
main() {
    int a, b, arr[50], result;
    char flag, symbol;

    a = 10;
    b = 20 + 5 * 2;

    if (a <= b) {
        result = a * b / 2;
        arr[1] = result % 3;
    } else {
        result = 0;
    }
}
6CSEC2@debian:~/Documents/230905252/Lab8$ ./exe/parser example.c
<main,3,1><(,3,5><),3,6><{,3,8><int,4,5><ID,4,9><,,4,10><ID,4,12><,,4,13><ID,4,15><[,4,18><NUM,4,19><],4,21><,,4,22><ID,4,24><;,4,30><char,5,5><ID,5,10><,,5,14><ID,5,16><;,5,22><ID,7,5><=,7,7><NUM,7,9><;,7,11><ID,8,5><=,8,7><NUM,8,9><+,8,12><NUM,8,14><*,8,16><NUM,8,18><;,8,19><if,10,5><(,10,8><ID,10,9><=,10,11><ID,10,14><),10,15><{,10,17><ID,11,9><=,11,16><ID,11,18><*,11,20><ID,11,22></,11,24><NUM,11,26><;,11,27><ID,12,9><[,12,12><NUM,12,13><],12,14><=,12,16><ID,12,18><%,12,25><NUM,12,27><;,12,28><},13,5><else,13,7><{,13,12><ID,14,9><=,14,16><NUM,14,18><;,14,19><},15,5><},16,1>
Parsing Successful

```